



simpler.

A programming language whose only goal is to be simple.

Language sketch · v0.1

Public domain (Unlicense)

Contents

1. The core law	1
2. Punctuation, in full	1
3. Operators are sugar over messages	2
4. Places are bidirectional	2
5. Effects live in the signature, not the syntax	3
6. Memory: value semantics, no GC	3
7. Capabilities replace imports	4
8. Control flow is messages	4
9. One language at every level	5
10. Compiled and fast	5
11. Why an AI writes it well	5
A complete small program	6
Locked decisions	7
Open next steps	7

Simple to learn, simple to read, simple for an AI to write, simple to extend, simple to install, simple to upgrade. Compiled, and fast. The design rule is a budget you enforce by refusal: every feature must pay for itself in problems solved, or it does not go in.

1. The core law

Everything is a message send.

```
receiver.name(arguments)
```

A field read, a method call, an operator, a property set: all of them are one thing, a message sent to a receiver. No-argument sends drop the parentheses.

```
screen.resolution      // read a value
file.read("report.txt") // call with an argument
items.length           // looks like a field, is a message
```

There is essentially one grammar production. That is the whole point.

2. Punctuation, in full

Three symbols, three meanings, zero overlap. Learn these and you have learned the syntax.

Symbol	Means	Example
=	bind a value to a name	x = 3
:	ascribe a type	x : Int
{ }	delimit a block	each { item -> ... }
.	send a message	screen.resolution

= has **one** meaning everywhere: bind this value to this name. The same act covers locals, places, and named arguments.

```
text = report.contents           // bind a local
screen.resolution = 1600, 1200   // bind a place (a setter)
mail.send(to = "boss@co", body = text) // bind a named argument
```

Assignment is a **statement**, not an expression. So `to = "x"` inside parentheses can only be a named argument, never a hidden assignment. A bonus falls out: `if a == b` can never be a mistyped `=`, because `=` cannot appear inside an expression at all.

3. Operators are sugar over messages

Exactly seven operators exist as sugar, and nothing more:

```
+ - * / == < >
```

Each desugars to a message: `a + b` is `a.+(b)`, `a < b` is `a.<(b)`. Everything else is a plainly named message, which keeps the surface small and removes the “what does this symbol mean” guessing game.

```
a.mod(b)           // no % operator
a.le(b)            // no <= operator
total.max(cap)     // reads like English, needs no symbol
```

4. Places are bidirectional

A field access is a first-class **place** with a getter and a setter, so a setter is never written by hand. The place’s *type* declares whether touching it touches the world.

```
point.x           : Place[Int]           // pure
```

```
screen.resolution : Place[Res] !Hardware !Fail // effectful
```

- A **pure** place is value semantics. `point.x = 5` produces an updated copy and rebinds your local. No mutation, no aliasing surprises.
- An **effectful** place mutates the world, and its type says so. `screen.resolution = 1600, 1200` performs a real mode switch that can fail.

The same `name = value` syntax means “functional update” for a pure place and “perform a typed effect” for an effectful one. The type tells you which. Effects on a deep path are the union of its links, computed by the compiler, so nothing can hide an effect inside `a.b.c = v`.

5. Effects live in the signature, not the syntax

Pure by default. Anything that touches the world is marked in the type: `!IO`, `!Fail`, `!Hardware`, and so on. A pure function cannot call an effectful one without declaring the effect, so cost stays visible at every call site. `?` propagates a failure.

```
read_all(file) !IO !Fail {  
  text = file.contents?      // ? hands the failure upward  
  text  
}
```

This is soft command-query separation: queries read the same as commands, but the type distinguishes them, so the convenience of uniform syntax never hides a cost or a failure.

6. Memory: value semantics, no GC

Value semantics by default, arenas for dynamic lifetime. No garbage collector (so no pauses and no idle battery drain) and no borrow checker (so nothing hard to learn). Pure assignment copies; the compiler elides the copy when it can prove nothing else shares the value. Real-world mutation happens only through effectful places, which the type system already tracks.

Why no collector. A GC is a second runtime you cannot hold in your head, so it breaks the simplicity promise where you cannot see it. It also works on its own schedule, waking to mark and sweep even when idle, which is the battery cost the design exists to avoid. Neither tax is necessary here, because the type system already did the hard part: value semantics gives each value one owner, and effects make every mutation visible, so the compiler can place every free at compile time. Arenas catch the rest, freeing a whole batch at once.

The trade-off is honest. Tangled, long-lived graphs are where a GC earns its keep; Simpler handles those with an explicit arena or handle, betting they are rare enough not to justify an always-on cost for every program. No GC, and no borrow checker either: value semantics buys memory safety without the tax of either.

7. Capabilities replace imports

There are no globals and no import system. `main` receives the world, and you pass subsets of it down. A signature becomes a permission list: a function can only touch what it was handed.

```
main(sys) {
    report = sys.files.open("report.txt")
    notify(sys.mail, report.contents?)
}

notify(mail, message) {           // can ONLY mail; never handed files
    mail.send(to = "boss@co", body = message)
}
```

Libraries are objects you receive, not namespaces you pull in:

```
json = sys.load("json")
data = json.parse(text)
```

Testing needs no mocking framework: pass a fake. Two versions of a library are just two objects you hold, so most of dependency hell disappears.

8. Control flow is messages

No control-flow keywords exist underneath. Blocks are `{ ... }` values passed as arguments; a block that takes a parameter names it before `in`, as `{ item in ... }`. A small amount of sugar keeps it readable.

```
// sugar
if user.active {
    screen.print("welcome")
} else {
    screen.print("denied")
}
```

```
// desugars to a plain message send
user.active.if(then = { screen.print("welcome") },
              else = { screen.print("denied") })

// iteration is a message too
items.each { item in
  screen.print(item.name)
}
```

9. One language at every level

Types are values, macros are ordinary functions, and the build script is a program, all written in this same syntax and evaluated at the stage where they belong. Because every expression is a message send, the AST is just a tree of message-send objects, so code is data with no parentheses ceremony. Build this last; it is the easiest piece to over-engineer.

10. Compiled and fast

Static types resolve the great majority of sends to direct calls or inlined field reads, so the dynamic-looking model has static cost. The compiler transpiles to C and hands it to the system C compiler. That buys a mature optimizer, every platform for free, and a self-contained toolchain small enough for one person to maintain.

The whole toolchain is one binary: `simpler build | run | fmt | test`. A single canonical formatter means every file looks identical, which helps humans and AI equally. Install is one download; upgrade replaces one binary.

11. Why an AI writes it well

The opening promise included “simple for an AI to write.” Here is why it holds, and it is not because the language is new.

An AI writes good code in a mature language by recalling millions of examples. It has seen none of this one. So the only thing that matters is whether the whole language fits in the prompt. Simpler’s surface is small enough that it does: one grammar production, three symbols, seven operators. The spec is the corpus. The model holds the entire language at once instead of recalling the right idiom from ten thousand.

Three properties then do the work:

- **The signature is the whole world.** Capabilities replace imports, so a function can touch only what it was handed. The parameter list is an exhaustive permission set, which kills the most common failure: calling an API that is not there.
- **Errors are mechanical and local.** Effects live in the type, so a pure function that calls an effectful one is rejected at that line. The write-compile-correct loop gives unambiguous signals a model converges on fast.
- **Less hidden state.** Value semantics removes the aliasing and shared mutation that AI-written code gets subtly wrong.

The same traits make the code efficient, not only correct. Cost is visible in the signature: `!IO, !Hardware, !Fail` say which paths touch the world, so a model optimising for battery can read the hot path without running it. And the division of labour is right. The model writes simple code; the compiler elides copies and frees arenas. It need not be a memory expert to get fast output.

The honest limit is the cold start. With no idiom fluency, an AI will not beat its Python on day one. But the gap is mostly memorised breadth, and there is little breadth here to memorise, so it closes fast. The real work is libraries: a received object must ship its signatures in a form a model can read. Given the design, the signature alone is enough.

A complete small program

```
// Read a report and mail it. The signature of every function
// states exactly which powers it holds.

main(sys) {
    report = sys.files.open("report.txt")
    if report.size > 0 {
        notify(sys.mail, report.contents?)
        sys.screen.print("sent")
    } else {
        sys.screen.print("empty report")
    }
}

notify(mail, body) {
    mail.send(to = "boss@co", subject = "Daily report", body = body)
}
```

Locked decisions

- Everything is a message send; one grammar production.
- `=` binds, `:` types, `{ }` blocks, `.` sends. No overlap.
- Assignment is a statement, not an expression.
- Operator sugar is exactly `+` `-` `*` `/` `==` `<` `>`; nothing else.
- Bidirectional places; setters are never hand-written.
- Effects and fallibility live in the type, not the syntax (soft CQS).
- Value semantics plus arenas; no GC, no borrow checker.
- Capabilities replace imports; no globals.
- Blocks use braces; a block parameter is named before `in` (no `->`).
- Compile by transpiling to C; one-binary toolchain.

Open next steps

- Pin the exact effect vocabulary (`!IO`, `!Fail`, `!Hardware`, ...).
- Decide whether pure places allow any sharing, or are strictly copied.
- Sketch the object/type definition form in full.
- Write the bootstrap compiler (emit C), then self-host.

By Geir Isene. Public domain (Unlicense).